Lời giải Quiz: Data Warehouse, Data Lake và Data Lakehouse

(Data Architecture)

Tác giả 1 **Tác giả 2**

Proofreading:

Đáp án: 1c, 2b, 3d, 4a, 5c, 6b, 7a, 8a, 9d, 10c, 11b, 12a, 13b, 14d.

1. Một công ty lưu dữ liệu đơn hàng thành các file Parquet dưới prefix `s3://shop/orders/`. Spark và Trino đọc "bảng" `orders` bằng cách liệt kê mọi file dưới prefix này rồi tự suy ra schema. Hệ thống không có catalog hay transaction log nào. Theo tiêu chí "lớp nào sở hữu khái niệm bảng", đây là kiến trúc gì?

a) Data Warehouse, vì dữ liệu được lưu ở định dạng columnar.
b) Data Lakehouse, vì nhiều engine cùng đọc một bản dữ liệu.
c) Data Lake, vì không lớp nào sở hữu bảng; "bảng" chỉ là một prefix thư mục.
d) Data Lakehouse, vì footer của file Parquet đã chứa schema.

Đáp án: c)

Engine tự liệt kê prefix và tự suy ra schema, nghĩa là không có catalog hay table format nào định nghĩa bảng. Đó là đặc điểm của Data Lake. Columnar (a) chỉ là cách lưu file. Việc nhiều engine cùng đọc (b) cũng xảy ra ở Data Lake. Schema trong footer Parquet (d) chỉ mô tả từng file, không mô tả bảng: nó không cho biết những file nào thuộc bảng hay phiên bản nào là hiện hành.

2. File `trips.parquet` có 4 row group. Footer cho biết khoảng giá trị của cột `fare_amount` trong từng row group: RG1 $[2, 45]$, RG2 $[3, 60]$, RG3 $[5, 480]$, RG4 $[1, 38]$. Với truy vấn `SELECT avg(tip_amount) FROM trips WHERE fare_amount > 100`, một engine hỗ trợ projection pushdown và predicate pushdown sẽ đọc những gì?

a) Cả 4 row group, đọc tất cả các cột.
b) Chỉ RG3, và chỉ hai cột `fare_amount`, `tip_amount`.
c) Chỉ RG3, nhưng đọc tất cả các cột.
d) Cả 4 row group, nhưng chỉ đọc cột `tip_amount`.

Đáp án: b)

Predicate pushdown bỏ qua RG1, RG2 và RG4 vì giá trị lớn nhất của chúng đều nhỏ hơn 100. Chỉ RG3 (max = 480) có thể chứa dòng thỏa điều kiện. Projection pushdown chỉ đọc các cột truy vấn cần: `fare_amount` để lọc và `tip_amount` để tính trung bình. Phương án (d) sai vì vẫn phải đọc `fare_amount` để kiểm tra điều kiện từng dòng trong RG3.

3. Một job upstream ghi nhầm một file có cột `VendorID` kiểu chuỗi vào cùng prefix của Data Lake, nơi các file khác đều lưu `VendorID` kiểu số nguyên. Lỗi này thường được phát hiện vào lúc nào?

a) Ngay lúc ghi, vì object storage kiểm tra schema của file.
b) Ngay lúc ghi, vì Parquet không cho ghi chuỗi vào một cột số.
c) Không bao giờ, vì engine tự bỏ qua các file sai kiểu.
d) Khi một engine quét prefix đó, thường là trong truy vấn của một người khác.

Đáp án: d)

Data Lake theo nguyên tắc schema-on-read: không ai kiểm tra schema khi ghi. Object storage chỉ lưu byte (a sai). File Parquet hợp lệ với schema riêng của nó, trong đó `VendorID` là chuỗi (b sai). Lỗi chỉ lộ ra khi engine đọc glob và cố ép kiểu, như lỗi `ConversionException` ở phần thực hành. Engine không tự bỏ qua file sai (c sai), mà làm hỏng cả truy vấn.

4. Một job ghi dữ liệu tháng 2 thành 4 file và bị dừng sau khi ghi xong file thứ hai. Một reader truy vấn ngay sau đó. Kết quả nào đúng?

a) Data Lake: reader thấy 2 file đã ghi mà không có cảnh báo. Iceberg: reader vẫn thấy snapshot cũ vì con trỏ trong catalog chưa đổi.
b) Data Lake: reader báo lỗi thiếu file. Iceberg: reader thấy 2 file đã ghi.
c) Cả hai đều thấy 2 file đã ghi, vì object storage có strong consistency.
d) Cả hai đều không thấy file nào, vì object storage chỉ hiển thị file khi job kết thúc.

Đáp án: a)

Ở Data Lake, "bảng" là mọi file dưới prefix, nên 2 file dở dang lập tức thuộc về bảng. Phần thực hành cho thấy reader thấy đúng 50% số dòng mà không có cảnh báo. Ở Iceberg, data file mới chỉ được nhìn thấy khi metadata trỏ tới chúng và con trỏ trong catalog được đổi ở bước commit. Job chết trước commit nên snapshot hiện hành không đổi, 2 file kia trở thành orphan file. Strong consistency (c) chỉ bảo đảm đọc thấy object vừa ghi, không bảo đảm tính nguyên tử của nhiều file.

5. Cho năm thành phần của một bảng Iceberg: (1) `snap-<id>.avro`, (2) `...-m0.avro`, (3) `00006-....metadata.json`, (4) `part-0.parquet`, (5) bản ghi của bảng trong catalog. Khi lập kế hoạch đọc bảng, engine đi qua chúng theo thứ tự nào?

a) (3) → (5) → (1) → (2) → (4)
b) (5) → (1) → (3) → (2) → (4)
c) (5) → (3) → (1) → (2) → (4)
d) (5) → (3) → (2) → (1) → (4)

Đáp án: c)

Catalog giữ con trỏ tới `metadata.json` hiện hành (5 → 3). `metadata.json` cho biết snapshot hiện hành và manifest list của nó (3 → 1). Manifest list liệt kê các manifest (1 → 2). Manifest liệt kê data file kèm thống kê cột (2 → 4). Phương án (d) đảo ngược manifest và manifest list: file `snap-*.avro` chính là manifest list, nằm trên manifest.

6. Writer A và writer B cùng đọc metadata phiên bản 7 của một bảng Iceberg, rồi mỗi bên ghi data file mới vào một partition khác nhau. A commit trước và thành công, con trỏ giờ trỏ tới phiên bản 8. Điều gì xảy ra khi B commit?

a) B ghi đè phiên bản 8, dữ liệu của A bị mất.
b) Phép compare-and-swap của B thất bại vì con trỏ không còn là 7. B đọc phiên bản 8, thấy không xung đột, tạo phiên bản 9 chứa cả hai thay đổi rồi commit lại.
c) B bị chặn cho tới khi A trả lại khóa (lock) trên bảng.
d) Catalog tự gộp hai file `metadata.json` của A và B thành một.

Đáp án: b)

Iceberg dùng optimistic concurrency: không khóa trước (c sai), xung đột được phát hiện lúc commit bằng compare-and-swap. CAS của B chỉ thành công nếu con trỏ vẫn là 7, nên B không thể ghi đè A (a sai). B phải tự đọc metadata mới, kiểm tra xung đột và tạo metadata mới; catalog không gộp metadata hộ (d sai). Vì hai writer ghi vào hai partition khác nhau nên không có xung đột dữ liệu và B commit lại thành công.

7. Đoạn code PyIceberg sau chạy trên một catalog trống. Giả sử lệnh `delete` xóa ít nhất một dòng. Sau khi chạy xong, `tbl.snapshots()` có bao nhiêu snapshot?

Code

```
tbl = catalog.create_table("silver.trips", schema=schema)
tbl.append(jan)
with tbl.update_schema() as u:
    u.add_column("tip_per_mile", DoubleType())
with tbl.update_spec() as s:
    s.add_field("pickup_at", DayTransform(), "pickup_day")
tbl.append(feb)
```

```
tbl.delete(delete_filter=GreaterThan("fare_amount", 500.0))
```

a) 3
b) 4
c) 5
d) 6

Đáp án: a)

Snapshot chỉ được tạo khi tập data file của bảng thay đổi. `create_table` tạo bảng rỗng, chưa có snapshot. `update_schema` và `update_spec` chỉ tạo `metadata.json` mới. Hai lệnh `append` và lệnh `delete` tạo ba snapshot với operation lần lượt là `append`, `append`, `overwrite`, đúng như output của `02_silver_lakehouse.py`. Số file `metadata.json` thì nhiều hơn số snapshot.

8. Tiếp tục câu 7. Khi đọc các dòng tháng 1 (được ghi trước lệnh `add_column`), cột `tip_per_mile` có giá trị gì?

a) NULL, vì file tháng 1 không chứa field ID của cột mới và không bị ghi lại.
b) Giá trị được Iceberg tự tính từ `tip_amount` và `trip_distance`.
c) Truy vấn báo lỗi schema mismatch cho tới khi ghi lại toàn bộ bảng.
d) 0, vì Iceberg điền giá trị mặc định của kiểu double.

Đáp án: a)

Iceberg ánh xạ cột qua field ID. Cột mới có field ID 16, không tồn tại trong file tháng 1, nên reader trả NULL; thêm cột là thao tác chỉ chạm metadata, không ghi lại file (c sai). Iceberg không hiểu ý nghĩa nghiệp vụ của cột nên không tự tính giá trị (b sai). Phần thực hành in ra `[None, None, None]` cho các dòng này, và lớp Gold dùng `coalesce` để tính lại.

9. Một bảng Iceberg ban đầu không có partition và đang chứa 1 file dữ liệu tháng 1. Đội dữ liệu thêm partition `day(pickup_at)` rồi append dữ liệu tháng 2 (28 ngày). Nhận định nào đúng?

a) Iceberg tự ghi lại file tháng 1 thành 31 file theo ngày.
b) Muốn được partition pruning, truy vấn phải lọc trên cột `pickup_day`.
c) File tháng 1 bị loại khỏi snapshot hiện hành vì không khớp partition spec mới.
d) File tháng 1 giữ spec cũ (`spec_id` 0), file tháng 2 theo spec mới (`spec_id` 1); điều kiện trên `pickup_at` vẫn loại được file ở cả hai spec.

Đáp án: d)

Partition evolution chỉ áp dụng cho dữ liệu ghi sau thay đổi; file cũ không bị ghi lại (a sai) và vẫn thuộc snapshot (c sai). Với hidden partitioning, người dùng lọc trên cột gốc `pickup_at`, còn engine tự quy đổi sang partition (b sai). File spec 1 được loại theo giá trị partition, file spec 0 được loại nhờ min/max của `pickup_at` trong manifest. Ở phần thực hành, truy vấn ngày 15/02 chỉ đọc 1/29 file.

10. Bảng `orders` (2 TB) nhận dữ liệu CDC từ database vận hành: cứ 5 phút lại có vài nghìn lệnh `UPDATE` rải rác khắp bảng. Dashboard đọc bảng vài lần mỗi giờ. Chiến lược nào phù hợp nhất?

a) Copy-on-Write, vì đọc luôn nhanh nhất.

b) Ghi lại toàn bộ bảng sau mỗi 5 phút để không phải dùng delete file.

c) Merge-on-Read (delete file hoặc deletion vector), kết hợp compaction định kỳ.

d) Bỏ table format, ghi thẳng file mới vào Data Lake.

Đáp án: c)

Cập nhật rải rác khắp bảng khiến Copy-on-Write phải ghi lại rất nhiều file lớn mỗi 5 phút (a sai), còn ghi lại 2 TB thì càng tệ hơn (b sai). Merge-on-Read chỉ ghi phần thay đổi; chi phí gộp lúc đọc chấp nhận được vì dashboard chỉ đọc vài lần mỗi giờ, và compaction định kỳ giữ cho số delete file không tăng mãi. Bỏ table format (d) đồng nghĩa với mất ACID và mất `UPDATE`.

11. Dùng PyIceberg 0.12, đội dữ liệu chạy `expire_snapshots` chỉ giữ snapshot hiện hành, rồi thấy dung lượng thư mục `data/` không giảm. Giải thích nào đúng và cần làm gì tiếp?

a) Lệnh expire chưa được commit; chạy lại lệnh là đủ.

b) Expire chỉ xóa snapshot khỏi metadata. Các file không còn được tham chiếu đã thành orphan file và cần được dọn riêng, với ngưỡng thời gian đủ xa để không xóa nhầm file của job đang ghi.

c) Các file vẫn cần cho time travel về những snapshot đã expire nên không được xóa.

d) Cần chạy compaction thì dung lượng mới giảm.

Đáp án: b)

Trong PyIceberg 0.12, `expire_snapshots` chỉ cập nhật metadata; phần thực hành cho thấy số snapshot giảm từ 302 xuống 1 nhưng thư mục `data/` vẫn có 301 file. Snapshot đã expire thì không time travel được nữa (c sai). Compaction (d) còn tạo thêm file mới. Bước dọn orphan file mới thu hồi dung lượng. Lưu ý: procedure `expire_snapshots` của Spark tự xóa các file chỉ thuộc snapshot đã expire, còn `remove_orphan_files` xử lý những file mà không metadata nào biết tới.

12. Dữ liệu khách hàng đến từ CRM và ERP với địa chỉ mâu thuẫn nhau. Cần khử trùng lặp, chọn bản ghi đáng tin cậy và lưu lịch sử thay đổi địa chỉ theo SCD Type 2. Trong kiến trúc Medallion, việc này nên đặt ở lớp nào?

a) Silver
b) Bronze
c) Gold
d) Trong chính hệ thống CRM và ERP

Đáp án: a)

Silver là nơi tạo ra thực thể nghiệp vụ đã làm sạch và thống nhất giữa các nguồn: chuẩn hóa schema, khử trùng lặp, áp quy tắc chọn bản ghi, SCD Type 2. Bronze phải giữ nguyên dữ liệu nguồn để có thể chạy lại pipeline (b sai). Gold dùng các thực thể đã sạch để dựng star schema và metric cho từng mục đích sử dụng (c sai). Hệ thống nguồn không thuộc phạm vi của nền tảng phân tích (d sai).

13. `fact_trips` có 5.848.609 dòng. Sau khi `INNER JOIN` với `dim_payment_type` (không có điều kiện `WHERE`), kết quả chỉ còn 5.712.662 dòng. Nguyên nhân khả dĩ nhất và cách phát hiện là gì?

a) DuckDB làm tròn số khi JOIN; cần đổi sang kiểu `DECIMAL`.
b) Một số `payment_type_key` trong fact không có trong dimension (ví dụ mã 0) nên bị `INNER JOIN` loại; phát hiện bằng test khóa ngoại mồ côi (`LEFT JOIN ... WHERE dim.key IS NULL`).
c) Snapshot Iceberg đã thay đổi giữa hai truy vấn.
d) Dimension có dòng trùng khóa nên JOIN làm giảm số dòng.

Đáp án: b)

Chênh lệch $5.848.609 - 5.712.662 = 135.947$ đúng bằng số chuyến có mã thanh toán 0 (*Flex Fare trip*) trong phần thực hành. `INNER JOIN` lặng lẽ loại các dòng không tìm thấy dimension tương ứng. Dimension trùng khóa (d) làm *tăng* số dòng chứ không giảm. Fact table là bảng DuckDB đã được tạo xong, không bị ảnh hưởng bởi snapshot Iceberg (c sai). Làm tròn số (a) không thay đổi số dòng.

14. Nhận định nào **không** phải là vấn đề của kiến trúc hai tầng (Data Lake + Data Warehouse) mà Armbrust và cộng sự nêu ra?

a) Dữ liệu trong warehouse trễ hơn nguồn hàng giờ đến hàng ngày.
b) Phải lưu hai bản dữ liệu và trả tiền compute cho ETL.
c) Thư viện ML khó đọc trực tiếp dữ liệu nằm trong warehouse.
d) Object storage không lưu được file Parquet.

Đáp án: d)

Ba vấn đề (a), (b), (c) lần lượt là dữ liệu cũ, tổng chi phí sở hữu và hỗ trợ hạn chế cho ML, cùng với độ tin cậy khi đồng bộ hai hệ thống. Object storage lưu được mọi loại file, kể cả Parquet; chính khả năng này là nền móng của Data Lake và Lakehouse.